5) Using Haffmann coding, arrive max encoding bits for phrase

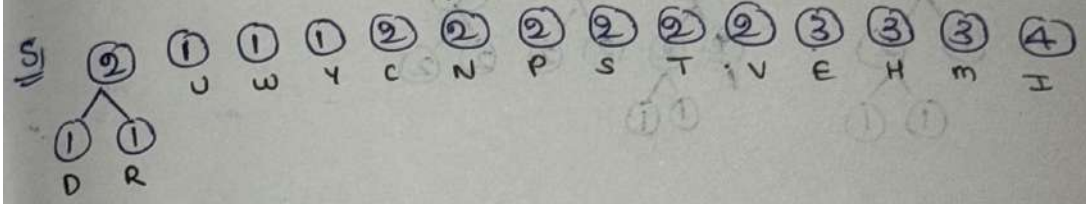
AMRITA VISHWA VIDYAPEETHAM CHENNAI CAMPUS

A	m	R	I	T	V	S	H	W	D	Y	P	E	C	N	U
7	3	1	4	2	2	2	3	1	1	1	2	3	2	2	1

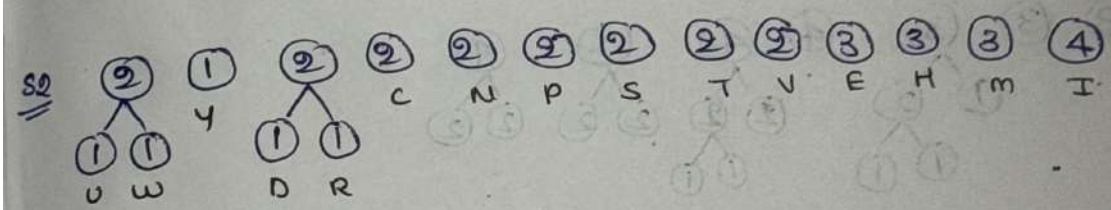
Arrange in ascending order

1 1 1 1 1 2 2 2 2 2 2 3 3 3 4
D R U W Y C N P S T V E H m I

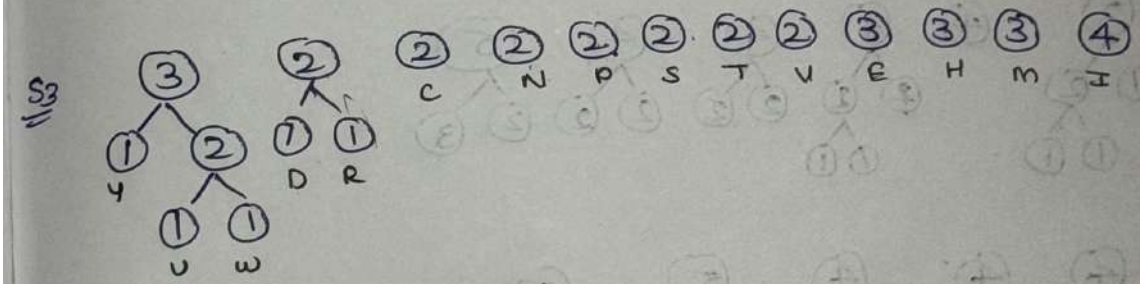
S1



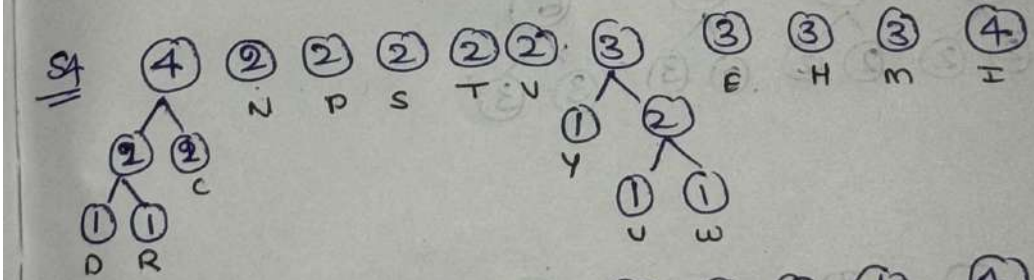
S2



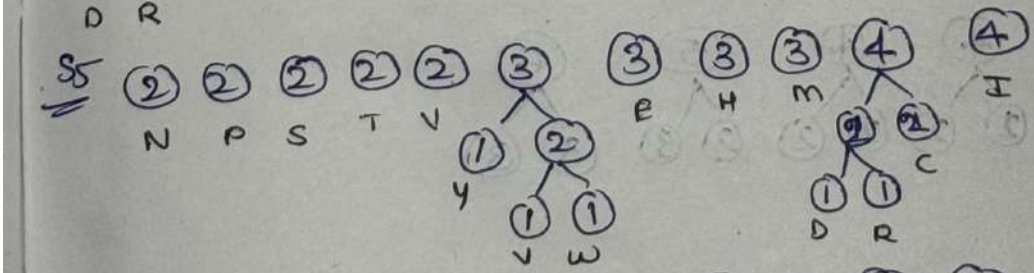
S3



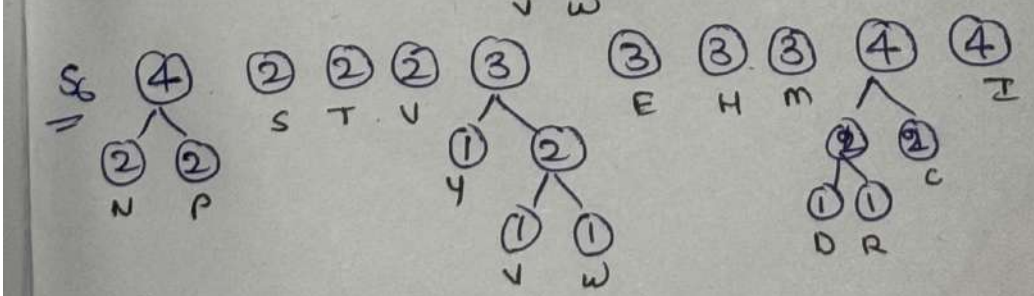
S4



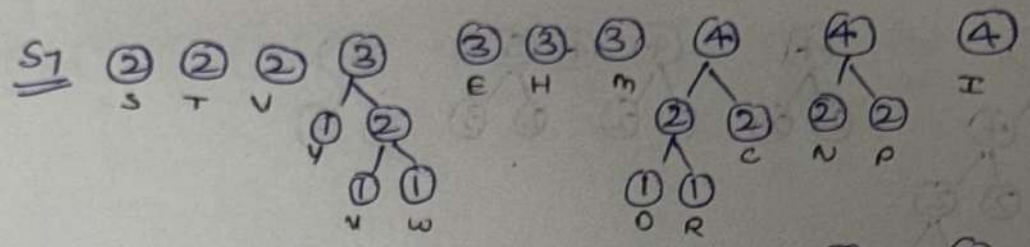
S5



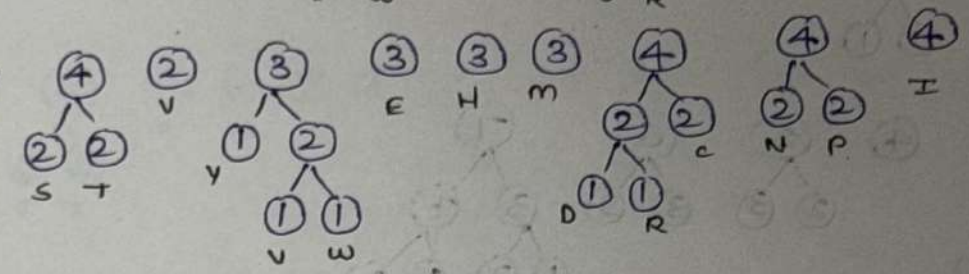
S6



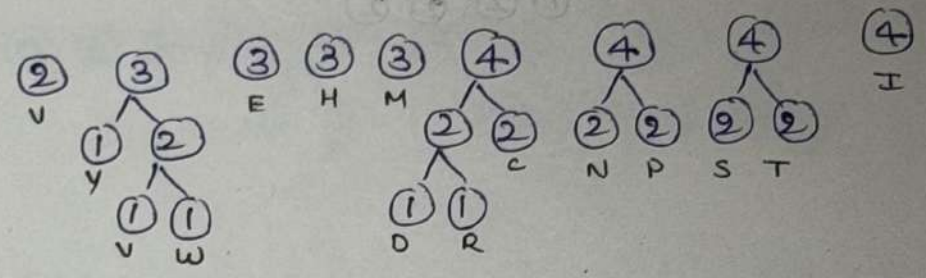
S7



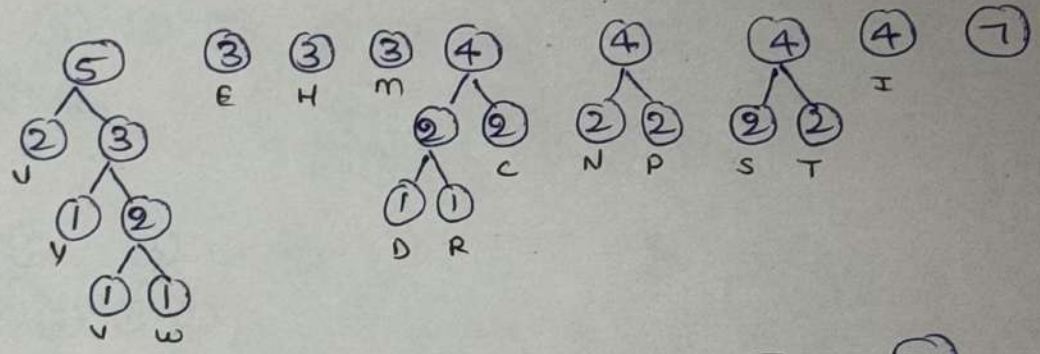
S8



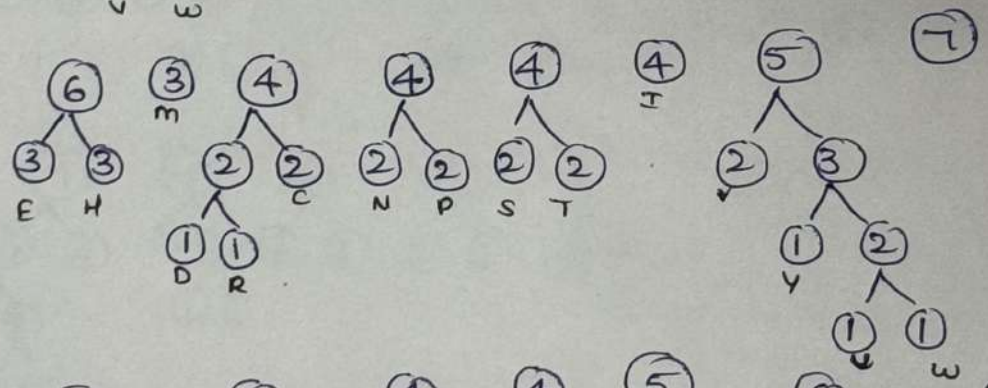
S9



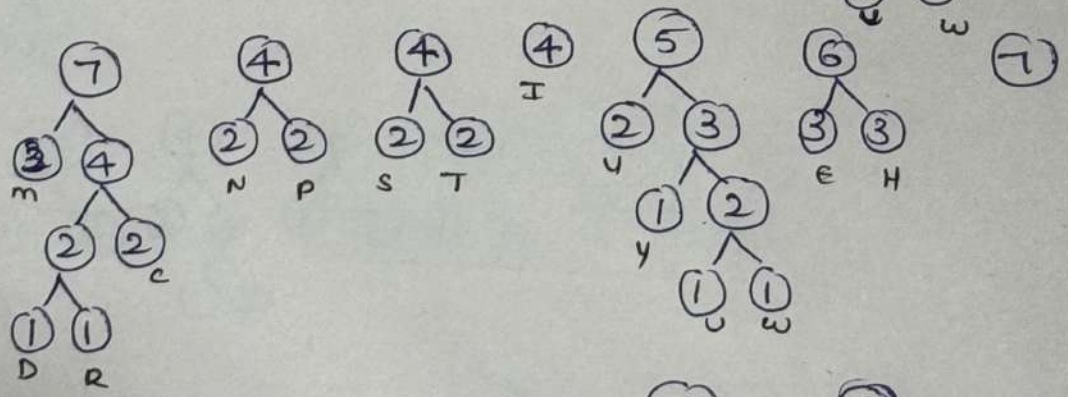
S10



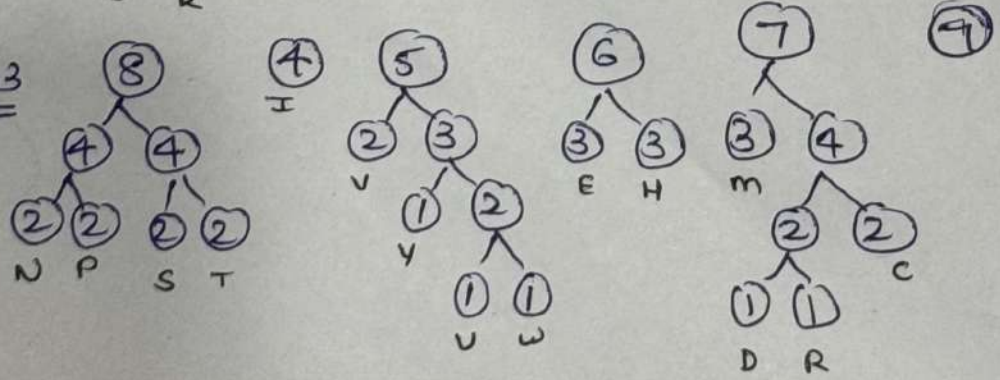
S11



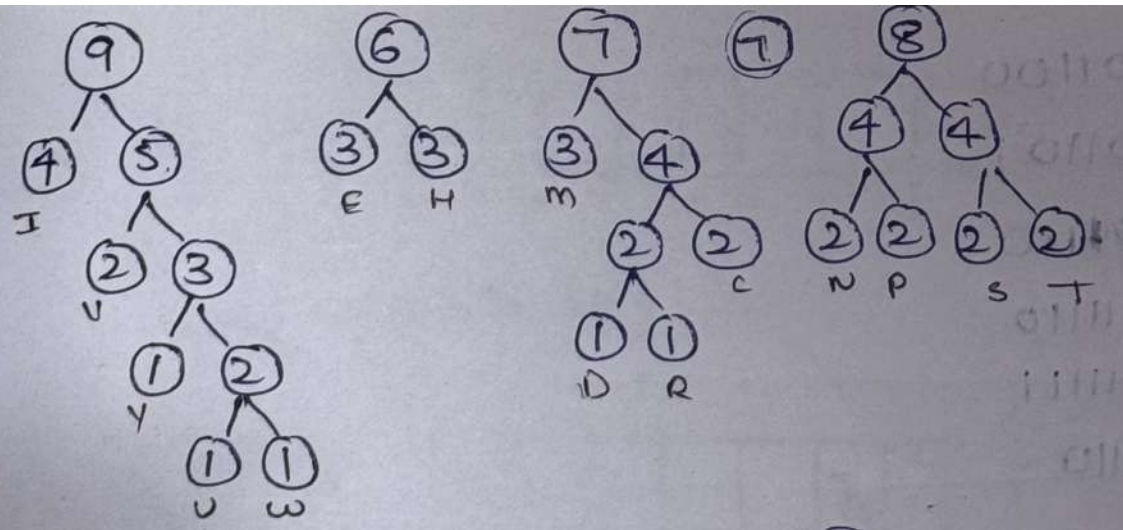
S12



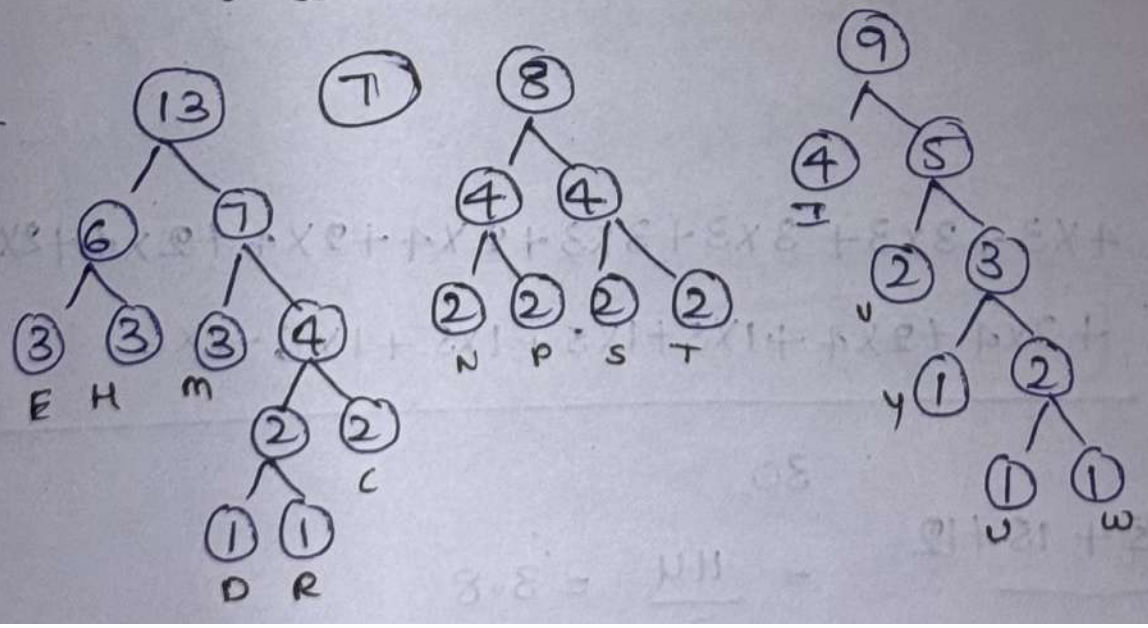
S13

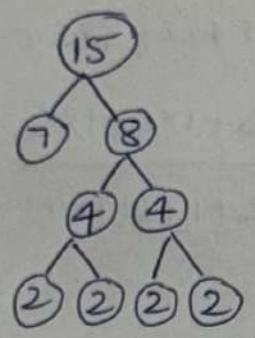


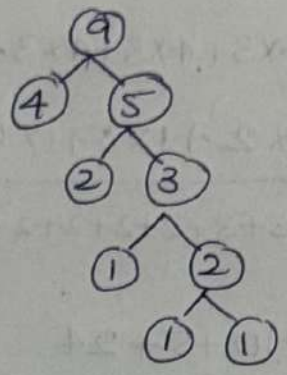
S14

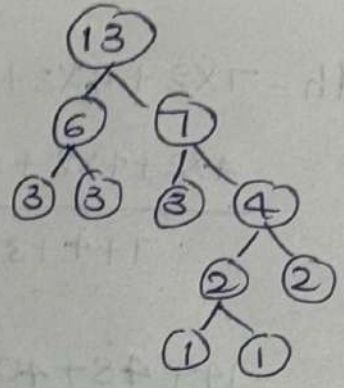

9
4 5
I
2 3
V
1 2
Y
1 1
U W
6
3 3
E H
7
3 4
M
2 2
C
1 1
D R
7
8
4 4
2 2 2 2
N P S T

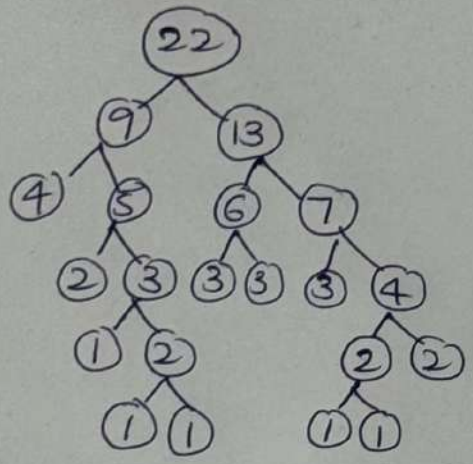
S15

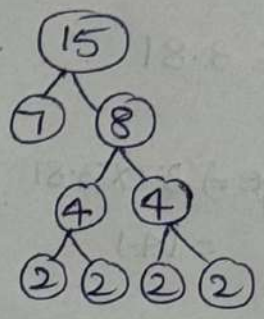

13
6 7
3 3 3 4
E H M
2 2
C
1 1
D R
7
8
4 4
2 2 2 2
N P S T
9
4 5
I
2 3
V
1 2
Y
1 1
U W

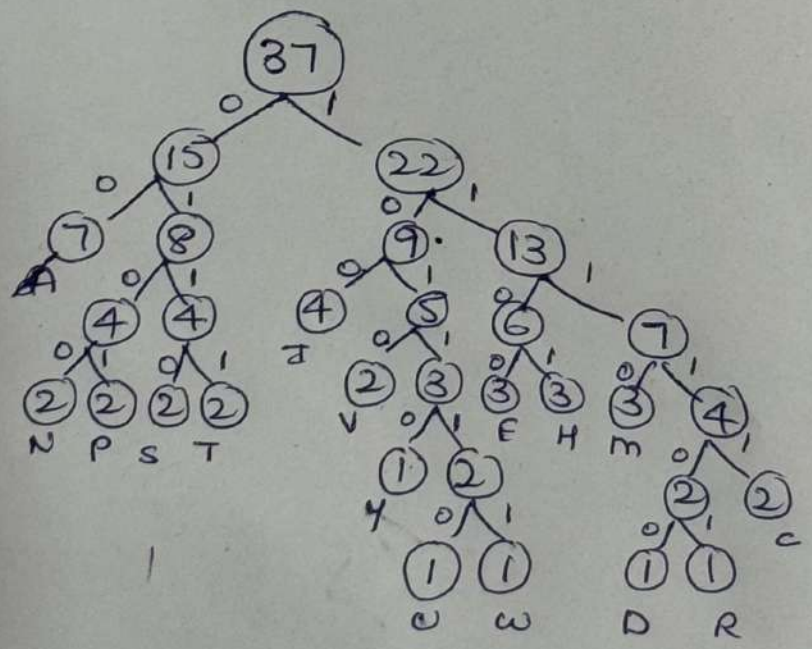












A - 00
I - 100
V - 1010
E - 1100
H - 1101
m - 1110
N - 0100
P - 0101
S - 0110
T - 0111

Y - 10110
U - 101110
W - 101111
D - 111100
R - 111101
C - 11111

$$\text{Avg code length} = \frac{7\times2+4\times3+4\times3+4\times3+4\times3+4\times2+4\times2+4\times2+4\times2+4\times2+5\times2+1\times5+1\times6+1\times6+1\times6+1\times6}{7+4+3+3+3+2+2+2+2+2+2+1+1+1+1+1}$$

$$= \frac{14+48+40+10+5+24}{37}$$

$$= \frac{141}{37} = 3.81$$

length of encoded message $\Rightarrow 37\times3.81$

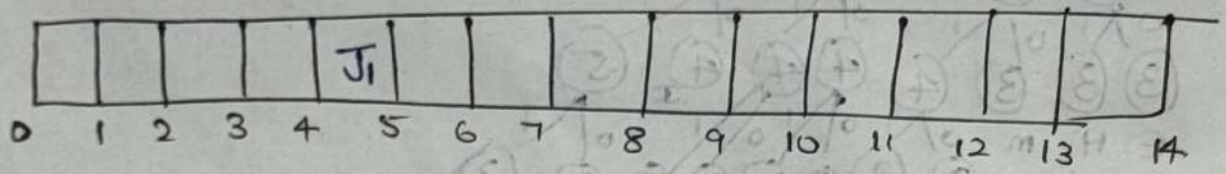
$= 141$

Job scheduling

Let there be 14 jobs for sequencing whose profits are 36, 21, 28, 19, 21, 13, 28, 25, 18, 20, 27, 22, 14, 26. Times are 5, 8, 3, 4, 4, 9, 12, 14, 2, 7, 5, 1, 6, 3.

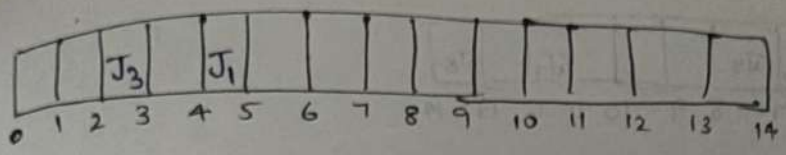
Sol Desending order P {36, 28, 28, 27, 26, 25, 22, 21, 21, 20, 19, 18, 14, 13}

T {5, 3, 12, 5, 3, 14, 1, 8, 4, 7, 4, 2, 6, 9}

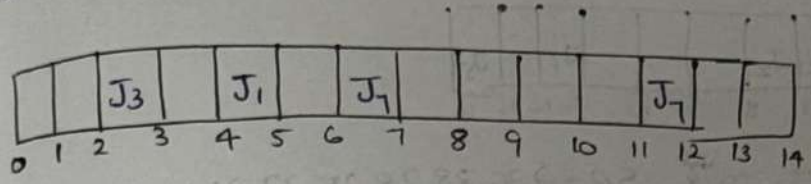


P = {36}

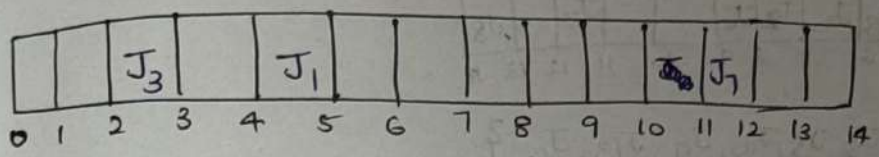
J = {J1}



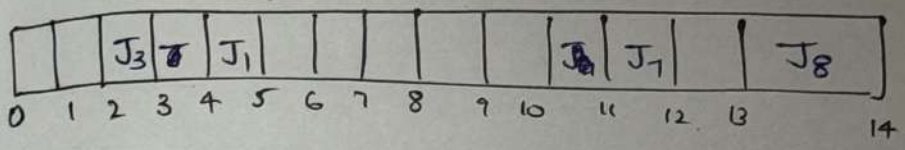
$S = \{J_1, J_3\}$ $Sp\ \{36, 28\}$



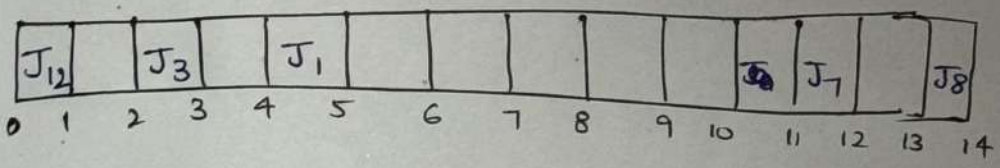
$S = \{J_1, J_3, J_7\}$ $Sp = \{36, 28, 28\}$



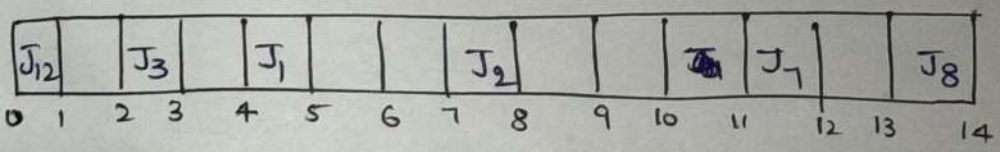
$S = \{J_1, J_3, J_7, J_{11}\}$ $Sp = \{36, 28, 28, 27\}$



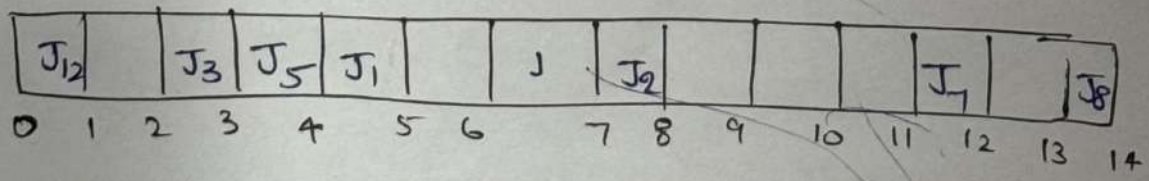
$S = \{J_1, J_3, J_7, J_{11}, J_8\}$ $Sp = \{36, 28, 28, 27, 25\}$



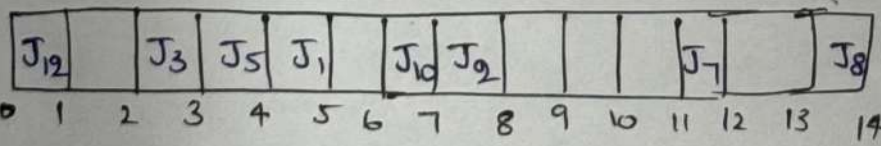
$S = \{J_1, J_3, J_7, J_{11}, J_8, J_{12}\}$ $Sp = \{36, 28, 28, 27, 25, 22\}$



$S = \{J_1, J_3, J_7, J_8, J_{12}, J_2\}$ $Sp = \{36, 28, 28, 25, 22, 21\}$



$S = \{J_1, J_3, J_7, J_8, J_8, J_{12}, J_2, J_5\}$ $Sp = \{36, 28, 28, 25, 22, 21, 21\}$



$S = \{J_1, J_3, J_7, J_8, J_{12}, J_2, J_5, J_{10}\}$ $Sp = \{36, 28, 28, 25, 22, 21, 21, 20\}$

J_{12}	J_9	J_3	J_5	J_1		J_{10}	J_2				J_7		J_8	
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

$S\ \{J_1, J_3, J_7, J_8, J_{12}, J_2, J_5, J_{10}, J_9\}$ $Sp = \{36, 28, 28, 25, 22, 21, 21, 20, 18\}$

J_2	J_9	J_3	J_5	J_1	J_{13}	J_{10}	J_2				J_7		J_8	
						6	7	8	9	10	11	12	13	14

$S = \{J_1, J_3, J_7, J_8, J_{12}, J_2, J_5, J_{10}, J_9, J_{13}\}$ $Sp = \{36, 28, 28, 25, 22, 21, 21, 20, 18, 14\}$

J_2	J_9	J_3	J_5	J_1	J_{13}	J_{10}	J_2	J_6			J_7		J_8	
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

$S = \{J_1, J_3, J_7, J_8, J_{12}, J_2, J_5, J_{10}, J_9, J_{13}, J_6\}$

$Sp = \{36, 28, 28, 25, 22, 21, 21, 20, 18, 14, 13\}$

Huffman Coding

CODE:

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

struct Node {
char data;
unsigned freq;
struct Node *left, *right;
};

struct Node* newNode(char data, unsigned freq) {
struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
temp->left = temp->right = NULL;
temp->data = data;
temp->freq = freq;
return temp;
}

struct MinHeap {
unsigned size;
struct Node* arr[MAX];
};

void swap(struct Node** a, struct Node** b) {
struct Node* t = *a;
*a = *b;
*b = t;
}

void heapify(struct MinHeap* heap, int i) {
int smallest = i;
int left = 2*i + 1;
int right = 2*i + 2;
```

```
if (left < heap->size && heap->arr[left]->freq < heap->arr[smallest]->freq)
smallest = left;

if (right < heap->size && heap->arr[right]->freq < heap->arr[smallest]->freq)
smallest = right;

if (smallest != i) {
swap(&heap->arr[i], &heap->arr[smallest]);
heapify(heap, smallest);
}
}

struct Node* extractMin(struct MinHeap* heap) {
struct Node* temp = heap->arr[0];
heap->arr[0] = heap->arr[--heap->size];
heapify(heap, 0);
return temp;
}

void insertHeap(struct MinHeap* heap, struct Node* node) {
int i = heap->size++;
while (i && node->freq < heap->arr[(i-1)/2]->freq) {
heap->arr[i] = heap->arr[(i-1)/2];
i = (i-1)/2;
}
heap->arr[i] = node;
}

struct Node* buildTree(char data[], int freq[], int size) {
struct MinHeap heap;
heap.size = 0;

for (int i = 0; i < size; i++)
heap.arr[heap.size++] = newNode(data[i], freq[i]);

for (int i = (heap.size-1)/2; i >= 0; i--)
heapify(&heap, i);

while (heap.size > 1) {
struct Node* left = extractMin(&heap);
```

```
struct Node* right = extractMin(&heap);

struct Node* top = newNode('$', left->freq + right->freq);
top->left = left;
top->right = right;

insertHeap(&heap, top);
}

return extractMin(&heap);
}

void printCodes(struct Node* root, int arr[], int top) {
if (root->left) {
arr[top] = 0;
printCodes(root->left, arr, top + 1);
}

if (root->right) {
arr[top] = 1;
printCodes(root->right, arr, top + 1);
}

if (!root->left && !root->right) {
printf("%c: ", root->data);
for (int i = 0; i < top; i++)
printf("%d", arr[i]);
printf("\n");
}
}

int main() {
char data[] = {'A','B','C','D','E','F'};
int freq[] = {5,9,12,13,16,45};
int size = sizeof(data)/sizeof(data[0]);

struct Node* root = buildTree(data, freq, size);

int arr[MAX], top = 0;
printf("Huffman Codes:\n");
printCodes(root, arr, top);
```

```
return 0;
}
```

OUTPUT:

```
kantatejesh@KANTAs-MacBook-Air ~ % gc
c -o huffman huffman.c
kantatejesh@KANTAs-MacBook-Air ~ % ./
huffman
Huffman Codes:
F: 0
C: 100
D: 101
A: 1100
B: 1101
E: 111
```

Job Sequencing Problem

CODE:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
int id;
int deadline;
int profit;
} Job;

int compare(const void *a, const void *b) {
Job *j1 = (Job *)a;
Job *j2 = (Job *)b;
return j2->profit - j1->profit;
}
```

```
int main() {
int n;
printf("Enter number of jobs: ");
scanf("%d", &n);

Job jobs[n];

for(int i = 0; i < n; i++) {
printf("Enter deadline and profit for job %d: ", i+1);
scanf("%d %d", &jobs[i].deadline, &jobs[i].profit);
jobs[i].id = i + 1;
}

qsort(jobs, n, sizeof(Job), compare);

int maxDeadline = 0;
for(int i = 0; i < n; i++)
if(jobs[i].deadline > maxDeadline)
maxDeadline = jobs[i].deadline;

int slot[maxDeadline];
for(int i = 0; i < maxDeadline; i++)
slot[i] = -1;

int totalProfit = 0;

for(int i = 0; i < n; i++) {
for(int j = jobs[i].deadline - 1; j >= 0; j--) {
if(slot[j] == -1) {
slot[j] = jobs[i].id;
totalProfit += jobs[i].profit;
break;
}
}
}


printf("\nScheduled Jobs: ");
for(int i = 0; i < maxDeadline; i++)
if(slot[i] != -1)
```

```
printf("J%d ", slot[i]);

printf("\nTotal Profit = %d\n", totalProfit);

return 0;
}
```

OUTPUT:

```
kantatejesh@KANTAs-MacBook-Air ~ % gc
c -o jobsch jobsch.c
kantatejesh@KANTAs-MacBook-Air ~ % ./
jobsch
Enter number of jobs: 4
Enter deadline and profit for job 1:  4 20
Enter deadline and profit for job 2: 1 10
Enter deadline and profit for job 3: 1 40
Enter deadline and profit for job 4: 1 30

Scheduled Jobs: J3 J1
Total Profit = 60
```